

November 10-13, 2025, Toronto, Canada

35th IEEE International Conference on Collaborative Advances in Software and Computing (IEEE CASCON 2025)

CHORUS

Automating Linear Programming Code Generation from Natural Language

Tutorial Session



Tasnim Ahmed Dr. Salimur Choudhury

Presented by: Tasnim Ahmed

PhD Student, GOAL Lab, School of Computing, Queen's University, Canada

Former Applied Scientist Intern, P13N Deals, Amazon, California, US

Tutorial Agenda

What We will Learn Today

1. **Foundations** - Linear Programming basics & problem walkthrough
2. **The Challenge** - Expertise barriers & version changes
3. **CHORUS Methodology** - Our solution with 4 key innovations
4. **Live Demonstration** - 7-stage pipeline walkthrough
5. **Results & Wrap-up** - Performance & Conclusion
6. **Q&A**

Feel free to ask questions anytime!

Get the Code for the Live Demo

github.com/tasnim7ahmed/CHORUS-CASCON

```
git clone https://github.com/tasnim7ahmed/CHORUS-CASCON
```

Tutorial Outline

Linear Programming

Challenges

Our Solution

Live Demonstration

Results & Conclusion

What is Linear Programming?

Definition

Linear Programming (LP): A mathematical optimization method for finding the best outcome from a set of linear constraints and a linear objective function.

Key Components:

- **Decision Variables:** What we control ($x_1, x_2, \dots$)
- **Objective Function:** What we optimize (max / min)
- **Constraints:** Limitations on variables

Real-World Applications:

- Supply chain optimization
- Production planning
- Resource allocation
- Portfolio optimization
- Transportation logistics

General Mathematical Form

maximize/minimize: $c^T x$

subject to: $Ax \leq b$ (constraints)

Example Problem

A fishery wants to transport their catch. They can either use local **sled dogs** or **trucks**. Local **sled dogs** can take **100 fish per trip** while **trucks** can take **300 fish per trip**. The **cost per trip** for **sled dogs** is **\$50** while the cost per trip for a **truck** is **\$100**. The **budget** is at most **\$1000** and the **number of sled dog trips** must be less than the **number of truck trips**. Formulate an LP to maximize the number of fish that can be transported.

Example Problem

A fishery wants to transport their catch. They can either use local **sled dogs** or **trucks**. Local **sled dogs** can take **100 fish per trip** while **trucks** can take **300 fish per trip**. The **cost per trip** for **sled dogs** is **\$50** while the cost per trip for a **truck** is **\$100**. The **budget is at most \$1000** and the **number of sled dog trips must be less than the number of truck trips**. Formulate an LP to maximize the number of fish that can be transported.

Let's identify the LP components in this problem...

Example Problem

A fishery wants to transport their catch. They can either use local **sled dogs** or **trucks**. Local **sled dogs** can take **100 fish per trip** while **trucks** can take **300 fish per trip**. The **cost per trip** for **sled dogs** is **\$50** while the cost per trip for a **truck** is **\$100**. The **budget is at most \$1000** and the **number of sled dog trips must be less than the number of truck trips**. Formulate an LP to maximize the number of fish that can be transported.

1. Decision Variables:

What do we control?

- x_1 = number of sled dog trips
- x_2 = number of truck trips

2. Objective Function:

What do we want to maximize?

- Total fish = $100x_1 + 300x_2$
- So: **maximize** $100x_1 + 300x_2$

3. Constraints:

What are the limitations?

- Budget constraint:
 $50x_1 + 100x_2 \leq 1000$
- Trip count constraint:
 $x_1 < x_2$
- Non-negativity:
 $x_1, x_2 \geq 0$

Complete LP Formulation

Decision Variables:

- x_1 = sled dog trips
- x_2 = truck trips

Objective:

$$\max \quad 100x_1 + 300x_2$$

Constraints:

$$50x_1 + 100x_2 \leq 1000$$

$$x_1 < x_2$$

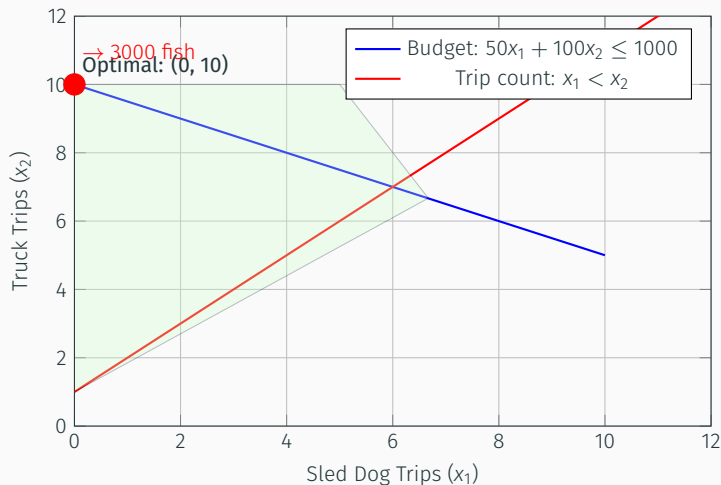
$$x_1, x_2 \geq 0$$

Geometric Interpretation:

- Each constraint defines a half-space
- Intersection forms the **feasible region**
- Optimal solution at a **corner point**

Let's visualize this problem...

Visualizing the Solution



How to Solve LP Problems?

Commercial & open-source software that implement optimization algorithms (e.g., Simplex, Interior Point methods)

Solver	Type	Performance	API Languages
Gurobi	Commercial	Excellent	Python, C++, Java, R
CPLEX	Commercial	Excellent	Python, C++, Java
FICO Xpress	Commercial	Excellent	Python, C++, Java
Hexaly	Commercial	Excellent	Python, C++, Java
HiGHS	Open-source	Good	C++, Python

Why Gurobi?

- Industry standard for large-scale problems
- State-of-the-art algorithms
- Extensive documentation & examples

Gurobi Python API: Core Components

Key Objects & Methods

Component	Gurobi Method/Object
Import library	<code>import gurobipy as gp</code>
Create model	<code>model = gp.Model("name")</code>
Add variables	<code>x = model.addVar(vtype=GRB.INTEGER, ...)</code>
Add constraints	<code>model.addConstr(expr, "name")</code>
Set objective	<code>model.setObjective(expr, GRB.MAXIMIZE)</code>
Optimize	<code>model.optimize()</code>
Get solution	<code>model.objVal, x.X</code>

Let's see the complete code for our Fishery problem...

Manual Gurobi Code for Fishery Problem

```
import gurobipy as gp
from gurobipy import GRB

m = gp.Model("fishery")
x1 = m.addVar(vtype=GRB.INTEGER, name="sled_dog_trips")
x2 = m.addVar(vtype=GRB.INTEGER, name="truck_trips")

m.addConstr(50*x1 + 100*x2 <= 1000, "budget")
m.addConstr(x1 <= x2 - 1, "trip_count")

m.setObjective(100*x1 + 300*x2, GRB.MAXIMIZE)
m.optimize()

print("Objective:", m.objVal)
print("Sled dogs:", x1.X, "Trucks:", x2.X)
```

Tutorial Outline

Linear Programming

Challenges

Our Solution

Live Demonstration

Results & Conclusion

The Accessibility Barrier

Writing Solver Code Requires Three Expertises

1. **Mathematical Modeling** - Formulate problem as LP/MIP
2. **Programming Skills** - Write syntactically correct code
3. **Solver-Specific API** - Know Gurobi methods & parameters

The Problem:

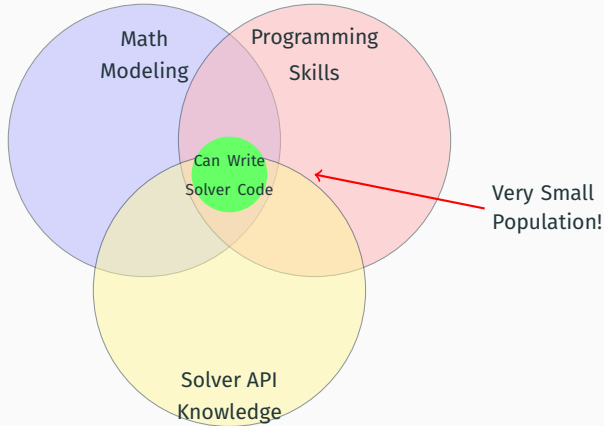
- Many domain experts lack programming backgrounds
- Operations researchers may not know Python/C++
- Gurobi documentation: 1000+ pages
- Learning curve: weeks to months

Impact

Limits who can leverage optimization technology

→ Reduces adoption in industry & education

The Expertise Intersection



Accessibility Bottleneck

Solver Versions Evolve

The Problem

Solver documentation & APIs change with every major version:

- Breaking syntax changes
- Deprecated methods
- New features & parameters
- Different import statements

Consequences:

- Fine-tuned LLMs become **obsolete**
- Documentation from 2023 $\neq$ Documentation from 2025
- LLMs trained on old code generate incorrect syntax
- RAG with outdated docs retrieves wrong examples

Example

Let's see real breaking changes between Gurobi versions...

Gurobi 9.x vs 11.x: Breaking Changes

Gurobi 9.x Code

```
# Version 9.x
import gurobi
from gurobi import *

model = Model()
x = model.addVars(3, name="x")

# Use quicksum
total = quicksum(x)

model.setObjective(total, GRB.
    MAXIMIZE)
model.optimize()
```

✓ Works in 9.x

Gurobi 11.x Code

```
# Version 11.x
import gurobipy as gp # Changed!
from gurobipy import GRB

model = gp.Model()
x = model.addVars(3, name="x")

# quicksum deprecated!
total = x.sum() # Use .sum()

model.setObjective(total, GRB.
    MAXIMIZE)
model.optimize()
```

✗ Old code fails!

What Actually Breaks?

Breaking Change #1: Import Statement

```
import gurobi  # Removed in 11.x!
```

Error: ModuleNotFoundError: No module named 'gurobi'

Breaking Change #2: quicksum with tupledict

```
x = model.addVars([(1,2), (3,4)])  
total = quicksum(x)  # Deprecated in 10.x, removed in 11.x
```

Error: DeprecationWarning / AttributeError

Why This Matters:

- LLMs trained on 2023 code → generate 9.x syntax
- Users with Gurobi 11.x → code fails to run
- Need: **Dynamic retrieval from current docs**

Traditional RAG Fails

Fixed-Size Chunking Issues:

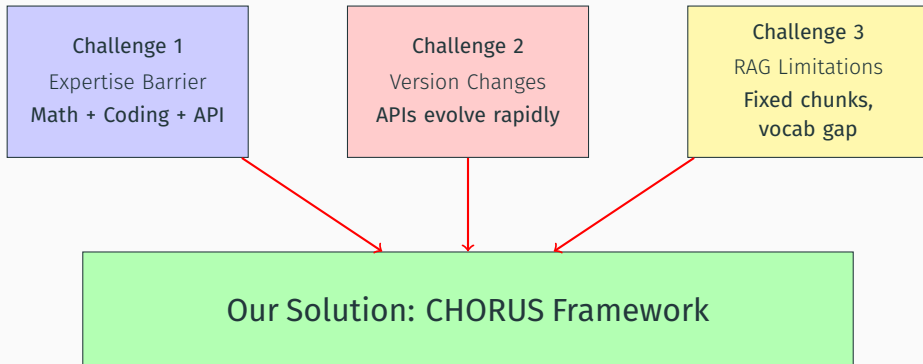
- Breaks functions mid-explanation
- Loses context (parent sections)
- No semantic coherence

Vocabulary Mismatch:

- User: "minimize cost"
- Code: `setObjective(..., GRB.MINIMIZE)`
- Lexical similarity $\approx 0!$
- Semantic meaning identical
- Traditional RAG misses this

We need a better approach...

The Triple Challenge



Tutorial Outline

Linear Programming

Challenges

Our Solution

Live Demonstration

Results & Conclusion

Why Traditional RAG Fails for LP Code Generation

Traditional RAG Pipeline:

Split documentation into fixed chunks (512 tokens) → Embed → Retrieve top-k → Feed to LLM

Critical Failures for Solver Code:

1. Fragmentation

- Functions split mid-explanation
- Code examples broken across chunks
- Context lost between sections

2. Vocabulary Mismatch

- Query: "minimize cost"
- Code: `GRB.MINIMIZE`
- Lexical similarity ≈ 0

3. Poor Precision

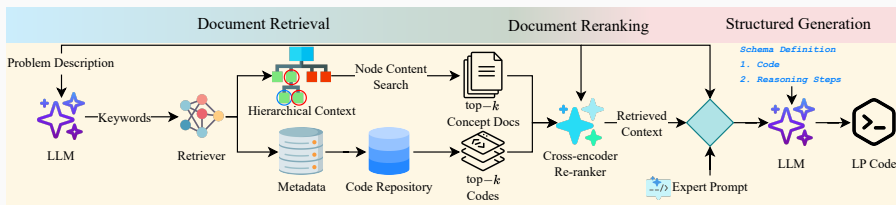
- Retrieves many irrelevant chunks
- Wrong API examples confuse LLM
- Domain mismatch (e.g., transportation vs production)

4. Unstructured Output

- LLM generates code without explaining
- No self-validation mechanism
- Higher hallucination rate

CHORUS

Core Insight: Technical domain RAG requires specialized document processing, not just better retrieval



Offline Phase (One-time):

- Parse Gurobi docs into tree structure
- Generate metadata for code examples
- Build dual vector stores
- Train cross-encoder reranker

Online Phase (Per Query):

- Extract problem keywords
- Dual retrieval: theory + code
- Two-stage reranking
- Structured code generation

Innovation 1: Hierarchical Chunking

Approach: Preserve document structure using tree-based representation

Traditional: Fixed 512 tokens

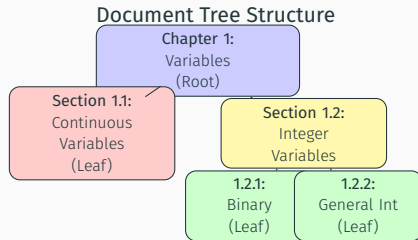
- Cuts mid-paragraph
- Fragments code examples
- Orphans explanations from context

CHORUS: Natural boundaries

- Chapter → Section → Subsection
- Complete semantic units
- Parent summaries prepended
- Adaptive size (max 400 words)

Why it works:

Mimics human documentation navigation—start broad, drill down to specifics



Example Leaf Chunk:

[Parent: "Integer variables represent countable quantities..."]
1.2.1 Binary Variables: Binary variables take values 0 or 1. Use `vtype=GRB.BINARY` when modeling yes/no decisions. Example: `x = model.addVar(vtype=GRB.BINARY)...`

Innovation 2: Metadata Augmentation

Approach: Generate LLM-based semantic metadata for code examples (inspired by HyDE)

The Vocabulary Gap:

Natural Language	Code Syntax
"minimize cost"	GRB.MINIMIZE
"link variables"	addConstr(y <= M*x)
"capacity limit"	sum(x) <= capacity

Generated Metadata (per code):

- Keywords (5-7):** Bridge terminology
Example: ["minimize", "setObjective", "transportation", "cost"]
- Synopsis (2-3 lines):** Semantic summary
Example: "Demonstrates cost minimization in transportation problems using Gurobi's setObjective with GRB.MINIMIZE..."

Retrieval: Match query to metadata, return original code

Example Code:

```
model = gp.Model()  
x = model.addVar(vtype=GRB.BINARY)  
y = model.addVar(vtype=GRB.INTEGER)  
model.addConstr(y <= 100*x)  
model.setObjective(500*x + 200*y,  
                  GRB.MAXIMIZE)
```

Auto-generated Metadata:

Keywords: maximize, binary, integer, capacity constraint, production

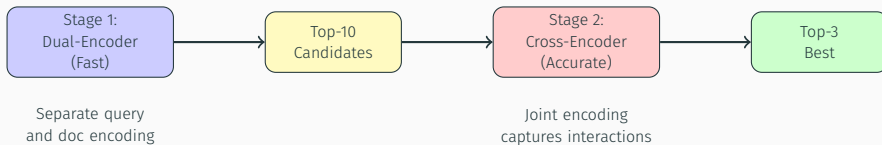
Synopsis: "Production planning linking binary factory decision with integer production quantity via capacity constraint"

User Query: "link binary and integer with capacity"

High metadata similarity → Code retrieved

Innovation 3: Cross-Encoder Reranking

Approach: Two-stage retrieval for speed + precision



Why Dual-Encoders First?

- Precompute all doc embeddings
- Fast cosine similarity search
- Scalable to large corpora
- **But: misses query-doc interactions**

Example miss:

Query: "binary OR integer"

May rank doc with "binary AND integer" equally

Why Cross-Encoders Second?

- Attention over concatenated query+doc
- Captures semantic relationships
- Accurate relevance scoring
- **But: too slow for initial retrieval**

Solution:

Use on small candidate set (10 → 3)

Best of both worlds

Innovation 4: Structured Reasoning

Approach: Force explicit reasoning through structured output schema

The Hallucination Problem:

LLMs can generate syntactically valid but semantically incorrect code:

- Undefined variables used
- Constraints omitted
- Wrong objective direction
- Type mismatches

Root Cause:

LLM jumps directly to code without validating logic

Solution:

Require LLM to justify each component before/during generation

Pydantic Schema:

```
class GurobiSolution(BaseModel):  
    reasoning_steps: str = Field(  
        description="Explain:  
        - Variable definitions  
        - Constraint mappings  
        - Objective correctness")  
  
    code: str = Field(  
        description="Complete  
        Gurobi Python code")
```

Self-validation: LLM catches errors during reasoning

CHORUS: Putting It All Together

Four Innovations Address Four Failures:

1. **Hierarchical Chunking**
→ Fixes fragmentation
Coherent, self-contained chunks
2. **Metadata Augmentation**
→ Bridges vocabulary gap
Semantic alignment with queries
3. **Cross-Encoder Reranking**
→ Improves precision
89% retrieval accuracy
4. **Structured Reasoning**
→ Prevents hallucination
Self-validation through explanation

Key Results (Preview):

Model	Base	CHORUS
Llama3.3 70B	22.9%	56.8%
Phi4 14B	19.4%	61.3%
Qwen2.5 32B	46.4%	59.9%
GPT-4	63.7% (baseline)	

Significance:

Open-source models match GPT-4 performance with CHORUS

Traditional RAG: 46-89% **worse** than CHORUS

Tutorial Outline

Linear Programming

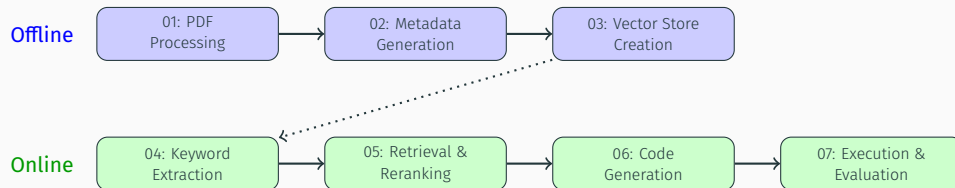
Challenges

Our Solution

Live Demonstration

Results & Conclusion

CHORUS Pipeline: Live Demonstration



Test Setup & Constraints

Full-document parsing → hierarchical chunking → vector store build is **compute- and time-intensive** (not feasible live).

In This Tutorial (Live Demo)

- Use a **small subset** of docs for chunking.
- Load a **smaller vector store**.
- Retrieval logic unchanged.
- Run inference via **OpenRouter API** using **Gemini 2.5 Flash**.
- Goal: **fast, reproducible** demo on a laptop.

Original Research Setup

- **Full** Gurobi docs & complete code example corpus.
- Two-stage retrieval + **cross-encoder reranking**.
- Open-source LLMs on server: *Llama3.1 (8B)*, *Llama3.3 (70B)*, *Phi4 (14B)*, *Qwen2.5-coder (32B)*.
- Objective: **max accuracy** & exhaustive evaluation.

Same pipeline behavior. **smaller corpus** ⇒ **faster demo** (no long indexing step). Everyone can **run locally** with an API key. Heavy compute **not required**.

Tutorial Outline

Linear Programming

Challenges

Our Solution

Live Demonstration

Results & Conclusion

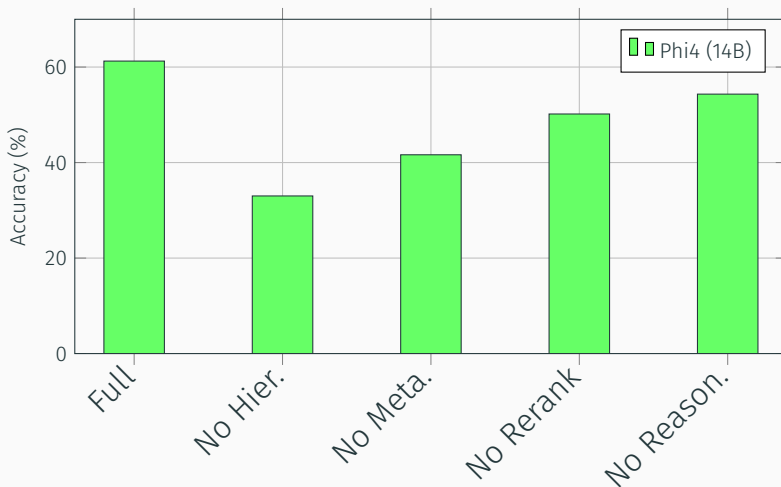
Performance Results: Open-Source LLMs vs GPT-4

Model (Size)	Baseline	CHORUS	Improvement
Phi4 (14B)	19.38%	61.25%	+216.0%
Llama3.3 (70B)	22.89%	56.75%	+147.9%
Qwen2.5-coder (32B)	46.44%	59.86%	+28.9%
Deepseek-r1 (32B)	10.73%	58.48%	+445.1%
GPT-3.5	52.60%	N/A	(baseline)
GPT-4	63.67%	N/A	(baseline)

Key Findings

- **Phi4 (14B)** with CHORUS matches GPT-3.5 performance
- **Qwen2.5 (32B)** with CHORUS approaches GPT-4 level
- **Zero-shot** - no fine-tuning required!

Ablation Study: Why Each Component Matters



Understanding the Ablations

1. **Hierarchical Chunking (-46%):**
 - Without: Fixed-size chunks break functions mid-explanation
 - Result: LLM gets incomplete context → incorrect code
2. **Metadata Augmentation (-32%):**
 - Without: Search "minimize cost" retrieves nothing (lexical mismatch)
 - Result: LLM lacks implementation examples
3. **Cross-Encoder Reranking (-18%):**
 - Without: Top-k contains irrelevant documents
 - Result: LLM gets distracted by noise
4. **Structured Reasoning (-11%):**
 - Without: LLM doesn't self-validate
 - Result: Subtle errors (missing constraints, wrong objective)

Limitations & Future Work

Current Limitations

- **Problem Scope:** Linear Programming only (not QP, MILP, MINLP)
- **Single Solver:** Gurobi only (not CPLEX, SCIP, etc.)
- **Documentation Updates:** Manual re-indexing when docs change
- **Execution Validation:** Windows signal timeout limitation

Future Directions:

1. **Expand Problem Types:** Quadratic, non-linear, constraint programming
2. **Multi-Solver Support:** CPLEX, SCIP, OR-Tools integration
3. **Incremental Updates:** Detect doc changes → auto re-index
4. **Interactive Refinement:** User provides feedback → regenerate code
5. **Benchmarking Suite:** Standardized evaluation across solvers

Broader Impact & Applications

Democratizing Optimization:

- **Domain Experts:** Supply chain managers, financial analysts without coding skills
- **SMEs:** Small businesses can leverage optimization without hiring specialists
- **Education:** Students learn LP concepts without programming barrier
- **Rapid Prototyping:** Quickly test multiple formulations

Technical Contributions:

- **RAG for Technical Domains:** Blueprint for documentation-driven code generation
- **Version-Agnostic:** Retrieves from current docs → always up-to-date
- **Open-Source Enablement:** Proves small models can match GPT-4 with proper RAG

Key Message

Optimization should be accessible to everyone, not just programmers!

Thank You!

Questions & Discussion

Tasnim Ahmed

`tasnim.ahmed@queensu.ca`